Table 5-3. *Array comparison functions*

Function + params	Description
`array_udiff(arr1, arr2, ..., func)`	As `array_diff()`, compares elements using `func`
`array_udiff_assoc(arr1, arr2, ..., func)`	As `array_diff_assoc()`, compares elements using `func`
`array_diff_uassoc(arr1, arr2, ..., func)`	As `array_diff_assoc()`, compares keys using `func`
`array_diff_ukey(arr1, arr2, ..., func)`	As `array_diff_key()`, compares keys using `func`
`array_udiff_uassoc(arr1, arr2, ..., func1, func2)`	As `array_diff_assoc()`, compares elements using `func1`, keys using `func2`
`array_uintersect(arr1, arr2, ..., func)`	As `array_intersect()` compares elements using `func`
`array_uintersect_assoc(arr1, ..., arr2, func)`	As `array_intersect_assoc()` compares elements using `func`
`array_intersect_uassoc(arr1, ..., arr2, func)`	As `array_intersect_assoc()` compares keys using `func`
`array_intersect_ukey(arr1, arr2, ..., func)`	As `array_intersect_key()` compares keys using `func`
`array_uintersect_uassoc(arr1, arr2, ..., func1, func2)`	As `array_intersect_assoc()` compares elements using `func1`, keys using `func2`

As with the basic intersect and diff functions, there can be an arbitrary number of array arguments, the first one will be compared in turn to all the others (with the same precedence rules for duplicates). The final argument (or arguments where two callables are required) will be treated as the callback(s).

Now let's see a couple of them working to give you a feel for how to use them. And to indulge myself, I'm going to imagine a fantasy classic car garage. This garage uses a database to store cars that are advertised for sale, and a different database to keep track of cars that are out for repair. If a car is out for repair it can't be sold, so we have a little program that maintains the list of cars for sale based on current repair status data.